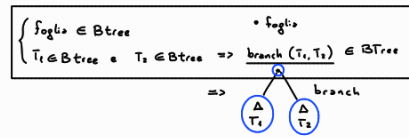


INDUZIONE STRUTTURALE

Definiamo BTree come



Definiamo $n: \text{BTree} \rightarrow \mathbb{N}$ conta i nodi di un albero binario

$$\begin{cases} n(\text{foglia}) = 1 \\ n(\text{branch}(T_1, T_2)) = 1 + n(T_1) + n(T_2) \end{cases}$$

Definiamo $h: \text{BTree} \rightarrow \mathbb{N}$ calcola l'altezza dell'albero

$$\begin{cases} h(\text{foglia}) = 0 \\ h(\text{branch}(T_1, T_2)) = 1 + \max(h(T_1), h(T_2)) \end{cases}$$

calcolo il valore massimo tra i due valori

Dimostrare per induzione strutturale che

② $\forall T \in \text{BTree}. n(T) \leq 2^{h(T)+1} - 1$

Base: Dobbiamo dimostrare ② per gli elementi: foglia

$T = \text{foglia}$ $n(T) = n(\text{foglia}) = 1$
 $h(T) = h(\text{foglia}) = 0$

Calcoliamo $2^{h(T)+1} - 1 = 2^{0+1} - 1 = 1$
 $n(T) = 1$ ✓

Passo induttivo: $T = \text{branch}(T_1, T_2)$

Supponiamo per Hp. ind. che

Hp. ind. $\rightarrow \begin{cases} \textcircled{2} n(T_1) \leq 2^{h(T_1)+1} - 1 \\ \textcircled{2} n(T_2) \leq 2^{h(T_2)+1} - 1 \end{cases}$

dimostriamo che $n(T) \leq 2^{h(T)+1} - 1$

$n(T) = 1 + n(T_1) + n(T_2)$ $h(T) = \max(h(T_1), h(T_2)) + 1$
 $\hookrightarrow h$

per def di n

$n(T) = 1 + n(T_1) + n(T_2) \leq 1 + 2^{h(T_1)+1} + 2^{h(T_2)+1} - 1$

↑
per ② e Hp. ind.

$\leq 2^h + 2^h - 1$
 $= 2^{h+1} - 1$
 $= 2^{h(T)+1} - 1$

BTree, definiamo

$\ell: \text{BTree} \rightarrow \mathbb{N}$ conta le foglie

$$\begin{cases} \ell(\text{foglia}) = 1 \\ \ell(\text{branch}(T_1, T_2)) = \ell(T_1) + \ell(T_2) \end{cases}$$

Dimostrare che $\ell(T) \leq 2^{h(T)} \quad \forall T \in \text{BTree}$

Base: $T = \text{foglia} : \ell(T) = 1 \quad 1 \leq 2^0$ ✓
 $h(T) = 0$

Passo induttivo: $T = \text{branch}(T_1, T_2)$

Hp. ind. $\ell(T_1) \leq 2^{h(T_1)} \quad \ell(T_2) \leq 2^{h(T_2)}$

Dobbiamo dimostrare che $\ell(T) \leq 2^{h(T)}$

$\ell(T) = \ell(T_1) + \ell(T_2)$ $h(T) = 1 + \max(h(T_1), h(T_2))$

$\ell(T_1) + \ell(T_2) \leq 2^{h(T_1)} + 2^{h(T_2)}$ $h = \max(h(T_1), h(T_2))$
 $\Rightarrow h(T_1) \leq h$
 $h(T_2) \leq h$

$\leq 2^h + 2^h = 2^{h+1} = 2^{h(T)}$

Exp (espressioni aritmetiche) definite induttivamente

Def di Exp

$$\begin{cases} n \in \text{Exp} \\ e_1, e_2 \in \text{Exp} \Rightarrow e_1 + e_2 \in \text{Exp} \quad e_1 - e_2 \in \text{Exp} \end{cases}$$

Def $\text{val}: \text{Exp} \rightarrow \mathbb{N}$ numero di numeri e parentesi in una exp

$$\begin{cases} \text{val}(n) = 1 \\ \text{val}(e_1 + e_2) = \text{val}(e_1) + \text{val}(e_2) + 1 \\ \text{val}(e_1 - e_2) = \text{val}(e_1) + \text{val}(e_2) + 1 \end{cases}$$

Def $\text{op}: \text{Exp} \rightarrow \mathbb{N}$ numero di operatori in Exp ($+$, $-$)

$$\begin{cases} \text{op}(n) = 0 \\ \text{op}(e_1 + e_2) = \text{op}(e_1) + \text{op}(e_2) + 1 \\ \text{op}(e_1 - e_2) = \text{op}(e_1) + \text{op}(e_2) + 1 \end{cases}$$

Dimostrare che $\text{val}(e) = \text{op}(e) + 1 \quad \forall e \in \text{Exp}$

Base: $e = n$ $\text{val}(e) = \text{val}(n) = 1$
 $\text{op}(e) = \text{op}(n) = 0 \quad 1 = 0 + 1$ ✓

Passo ind.: $e_1 + e_2$ (e_1, e_2 Analoghi)

Hp. ind. $\text{val}(e_1) = \text{op}(e_1) + 1$
 $\text{val}(e_2) = \text{op}(e_2) + 1$

Dobbiamo dim. che $\text{val}(e) = \text{op}(e) + 1$

$\text{val}(e) = \text{val}(e_1) + \text{val}(e_2) + 1$ $\text{op}(e) = \text{op}(e_1) + \text{op}(e_2) + 1$
Hp. ind.

$= \text{op}(e_1) + \text{op}(e_2) + 1 + 1 = \text{op}(e) + 1$

op(e) per def di op

$\begin{matrix} \text{main} \\ \left[\begin{array}{c} \text{A} \\ \text{B} \\ \text{C} \end{array} \right] \end{matrix}$
 $\begin{matrix} \text{A} \\ \text{B} \\ \text{C} \end{matrix}$
 $\begin{matrix} \text{A} \\ \text{B} \\ \text{C} \end{matrix}$

$\text{main} \rightarrow \text{B} \rightarrow \text{C} \rightarrow \text{A}$

mala

x	f
3	2

mala

chiamato *mala* → B

x	f
3	1

mala

linea dinamica

x	f
3	2
3	4

una statica

$N_0 = Sd(mala) - Sd(B) = 1$
 $= 0 + 1 \neq 0$

$C(S) = mala$

esplorazione di B

y = x + y
↳ livello + 2

$N_y = -Sd(mala) + Sd(B)$
 $\Rightarrow y\text{ mala} = 2$
 $= 0 + 1 \neq 0$

y = 2+2=4
nel mala

la più vicina alla catena di misurazione statico alle def.

uso y

Dobbiamo risolvere di L la catena statica per trovare il corretto riferimento a y

main

x	2
y	4

B

x	2
w	4

C

x	8
---	---

Line statico:

$$N_C = Sd(B) - Sd(c) + t$$
$$\downarrow$$
$$2 - 2 + t = 0$$
$$Sd(c) = 8$$

localc

$$X = y \cdot w$$

8

$$N_B = Sd(c) - Sd(B)$$
$$\downarrow$$
$$8 \text{ in } B \rightarrow 4$$
$$N_C = Sd(c) - Sd(main) = 2 \text{ in main}$$
$$\rightarrow 4$$

The diagram illustrates the execution of a program with nested loops and function calls, showing the state of the stack and the call graph.

Stack State:

- main:** Contains variables $x=1$ and $y=3$.
- B:** Contains variables $x=2$ and $y=4$.
- C:** Contains variables $x=3$ and $y=5$.
- A:** Contains variables $x=4$ and $y=5$.

Control Flow:

- A red arrow indicates the flow from **main** to **B**, then to **C**, and finally to **A**.
- A blue arrow indicates the flow from **A** back to **C**, then to **B**, and finally back to **main**.

Call Graph:

- main** calls **Sd(c)**.
- Sd(c)** calls **Sd(a)**.
- Sd(a)** calls **CS(cs(c))**.
- CS(cs(c))** calls **CS(b)**.
- CS(b)** calls **main**.

Local Variable:

- A blue circle labeled **local** contains the expression $z = x + y$.
- Arrows point from the x and y variables in the **A** frame to the $z = x + y$ expression.

Mathematical Expressions:

- $K_0 = Sd(c) - Sd(a) + 1$
- $CS(A) = CS(cs(c)) = CS(b) = main$
- $N_y = N_x = Sd(A) - Sd(main) = 1 \rightarrow main$

[illegible]

Diagram illustrating memory stack layout and return values:

- Variables and their addresses:
 - `x` at `0x8`
 - `y` at `0x4`
 - `z` at `0x0`
 - `w` at `0x4`
- Return values from functions:
 - `x = y + w` returns `0x8`
 - `y = x + z` returns `0x4`
 - `z = x + y` returns `0x4`
- Note: Execution C

Diagram illustrating the execution of the code snippet:

```
graph LR; x["x"] --> C8["C, 8"]; y["y"] --> min4["min, 4"]; z["z"] --> A12["A, 12"]; w["w"] --> B4["B, 4"]; C8 --> min4; min4 --> main["main, 1"]; A12 --> main; B4 --> main; main --> E["Escezione A"]; main --> Z["z = x + y"];
```

The diagram shows the flow of execution from the initial state to the final state, highlighting the exception handling mechanism. The variable `z` is updated to `x + y` after the exception is caught.

```

lista function ( lista list ) {
  if (length(list) == 0) then return list = (1)
  else return concat (function (cdr(list)), car(list))
}

function (3,6,9) (9,6,3)
  ↳ concat (function (6,9), 3) (9,6)
    ↳ concat (function (9), 6) (9)
      ↳ concat (function (11), 9)
        ↓
        (1)

La funzione inverte gli
elementi della lista

lista function (lista list, lista res) {
  if (length(list) == 0) return res;
  else return function (cdr(list), concat (car(list), res));
}

lista function (lista list) { return function (list, 1); }

function (3,6,9) (9,6,3)
  ↳ function ( (3,6), 1 )
    ↳ function ( (6,9), concat (3, 1) )
      ↳ function ( (9), concat (6, (3)) )
        ↳ function ( (1), concat (9, (6,3)) )
          (9,6,3)

```